

Laporan Praktikum Minggu Ke-4 Kontrol Cerdas

Minggu ke-4

Nama : Nisa Ayu Maslikhah

NIM : 224308064

Kelas : TKA 7C

Akun Github (Tautan): <https://github.com/nisaayumaslikhah>

1. Judul Percobaan

Reinforcement Learning for Autonomous Control

2. Tujuan Percobaan

Tujuan dari percobaan ini, sebagai berikut:

- Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
- Mengimplementasikan agen RL menggunakan algoritma Deep Q-Network (DQN).
- Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
- Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
- Menggunakan GitHub untuk version control dan dokumentasi praktikum.

3. Landasan Teori

Reinforcement learning mirip dengan banyak topik dengan nama yang berakhiran -ing, seperti *machine learning*, *planning*, and *mountaineering*, karena pembelajaran ini sekaligus merupakan masalah, kelas metode solusi yang efektif pada kelas masalah tersebut, dan bidang yang mempelajari masalah-masalah ini beserta metode solusinya (Sutton & Barto, t.t.). Reinforcement Learning (RL) adalah suatu metode pembelajaran mesin di mana sebuah agen belajar mengambil keputusan optimal melalui interaksi dengan lingkungan (environment). Agen menerima umpan balik berupa reward (penghargaan) atau penalty (hukuman) dari

tindakannya sehingga dapat mengoptimalkan strategi atau kebijakan guna mencapai tujuan tertentu. RL sangat cocok untuk sistem kendali otonom yang memerlukan adaptasi dan pengambilan keputusan secara real-time dalam lingkungan yang dinamis. Contohnya termasuk navigasi robot, kendaraan otonom, pengoptimalan rute, dan pengaturan proses dalam manufaktur atau logistik.

Salah satu algoritma RL yang populer adalah Deep Q-Network (DQN), yang mengkombinasikan Q-learning dengan jaringan saraf dalam (Deep Neural Networks) untuk mengaproksimasi fungsi nilai aksi. *Deep Q-Network (DQN)* adalah algoritma pembelajaran penguatan yang menggabungkan algoritma *Q-Learning* dan *Deep Neural Network* sebagai fungsi aproksimator (mendekati nilai sebenarnya). Dalam DQN, *Q-values* (nilai Q) digunakan untuk mengukur nilai tindakan dalam suatu keadaan. Jaringan saraf mendalam (*Deep Neural Network*) digunakan untuk memodelkan *Q-values* dalam DQN (Ridho Amanda Putra, t.t.). DQN memungkinkan agen melakukan pembelajaran dari pengalaman masa lalu yang disimpan dalam memori replay, dengan strategi eksplorasi dan eksploitasi yang seimbang melalui parameter epsilon-greedy. Agen DQN diterapkan pada environment simulasi CartPole menggunakan pustaka OpenAI Gym, yang bertujuan mengendalikan tiang pada kereta supaya tetap seimbang tanpa jatuh.

Simulasi lingkungan OpenAI Gym, khususnya environment CartPole-v1, untuk melatih dan menguji agen DQN. Selama pelatihan, agen mencoba mengontrol batang keseimbangan dengan belajar dari reward yang diperoleh, sambil meminimalkan kesalahan. Akhirnya, agen diuji dengan mengevaluasi performanya dalam mengontrol environment tersebut setelah pelatihan. Selain itu, simulasi pada lingkungan OpenAI Gym seperti MountainCar-v0 mengilustrasikan bagaimana agen DQN belajar dari trial-and-error dengan melakukan update jaringan secara iteratif, sehingga mampu mengoptimalkan keputusan kendali pada setiap state untuk menyelesaikan tugasnya dengan baik. RL sangat relevan untuk Sistem Kendali Otonom karena memungkinkan sistem untuk belajar mengontrol dinamika kompleks tanpa model eksplisit lingkungan. Agen belajar kebijakan kendali langsung dari interaksi dan umpan balik

4. Analisis dan Diskusi

➤ Analisis

- a. Bagaimana performa agen dalam mengontrol environment CartPole?

Performa agen dalam mengontrol CartPole diukur berdasarkan skor, dimana didapatkan skor mendekati maksimum 500, yang menunjukkan kemampuan menjaga tiang tetap seimbang diatas kereta dalam durasi panjang. Prosesnya menunjukkan adanya peningkatan yang signifikan reward dari episode ke episode hingga dapat menjadi optimal.

- b. Bagaimana perubahan parameter (misal: gamma, epsilon, learning rate) mempengaruhi kinerja agen?

- Pada gamma mempengaruhi seberapa jauh agen untuk mempertimbangkan reward masa depan, serta dapat meningkatkan kemampuan untuk memecahkan masalah dengan *delay reward*. Gamma yang tinggi akan membuat agen lebih berorientasi jangka Panjang dan lebih optimal.
- Pada epsilon agen lebih banyak melakukan eksplorasi (aksi acak). Epsilon yang terlalu besar, maka menyebabkan adanya eksplorasi berlebih serta pelatihannya lambat. Namun, jika epsilon kecil agen akan menggunakan strategi yang baik. Terdapat epsilon decay untuk mengatur penurunan nilai secara bertahap.
- Pada learning rate menentukan laju bobot jaringan saraf. Jika learning rate terlalu tinggi maka dapat menyebabkan pelatihan tidak stabil (bobot terlalu cepat), sedangkan jika terlalu rendah akan membuat pelatihan menjadi lambat. Nilai 0.001 sering digunakan sebagai nilai optimal balance.

- c. Apa tantangan yang muncul selama pelatihan agen RL?

Tantangannya adalah pelatihan yang tidak stabil terutama jaringan saraf yang dalam sehingga diperlukan penanganan seperti experience replay dan target network. Selain itu, Resiko overfitting pada data pelatihan dan ketidakmampuan generalisasi jika lingkungan berubah. Dan Menemukan

keseimbangan yang tepat antara eksplorasi (mencari potensi baru) dan eksploitasi (menggunakan yang sudah diketahui) sangat krusial.

➤ **Diskusi**

- a. Apa perbedaan utama antara Reinforcement Learning dan metode supervised learning dalam sistem kendali?

RL berfokus pada pengambilan keputusan berurutan dengan umpan balik reward, tanpa label output eksplisit pada setiap input—berbeda. Sedangkan supervised learning mempelajari fungsi mapping dari input ke output dengan data labeled. RL cocok untuk sistem kendali yang kompleks/dinamis serta dapat digunakan untuk belajar dari trial and error dan feedback lingkungan, sedangkan supervised learning cocok untuk sistem kendali karena data aksi benar (dari ahli) sudah tersedia.

- b. Bagaimana strategi eksplorasi (exploration) dan eksploitasi (exploitation) dapat dioptimalkan pada agen RL?

Eksplorasi dan eksploitasi diatur menggunakan epsilon-greedy, dengan nilai epsilon yang tinggi untuk eksplorasi, dan nilainya dikurangi (*decay*) secara bertahap seiring waktu. Selain itu, bisa menggunakan teknik seperti Upper Confidence Bound (UCB), atau model-based RL yang digunakan untuk mengira ngira reward sebelum dilakukan eksekusi. Hal tersebut dapat membuat agem untuk mencari Solusi baru sekaligus memanfaatkan kebijakan terbaik saat ini.

- c. Potensi aplikasi lain dari RL dalam sistem kendali nyata apa saja yang dapat diimplementasikan?

RL dalam sistem kendali dapat di implementasikan secara luas dengan membutuhkan keputusan sekuensial dan otonom antara lain pada kendaraan otonom digunakan untuk pengendalian rute dan manuver, selain itu robotika sangatlah adaptif untuk navigasi dan manipulasi. Dan pada pengendalian industry digunakan untuk pengaturan suhu, tekanan, atau sistem HVAC untuk meminimalisir konsumsi energi.

5. Assignment

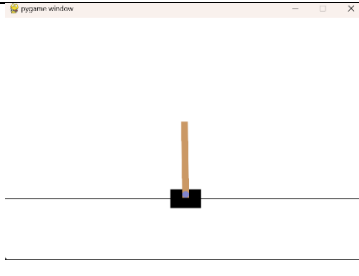
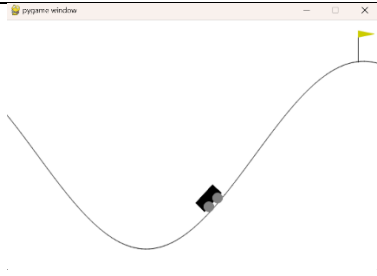
Pada praktikum kali ini, membuat sebuah program yang menerapkan algoritma Deep Q-Network (DQN) untuk menyelesaikan tantangan *MountainCar-v0* dan keseimbangan tiang CartPole dalam durasi waktu yang panjang dengan menggunakan pustaka *Gymnasium*. Agen DQN menggunakan jaringan saraf tiruan untuk mendapatkan fungsi nilai aksinya dan belajar melalui interaksi trial-error dalam lingkungan. Perangkat keras dan perangkat lunak modern seperti TensorFlow/Keras, dan menggunakan *Gymnasium* untuk simulasi, akan memberikan kerangka kerja yang efektif untuk diimplementasikan DQN. Pelatihan agen dilakukan dengan strategi epsilon-greedy untuk mengoptimalkan keseimbangan antara eksplorasi dan eksploitasi.

Program ini juga terintegrasi dengan *Pygame* untuk menangani input dari pengguna. Jika jendela permainan ditutup oleh pengguna, program akan berhenti secara otomatis. Namun, karena program menggunakan mode tampilan (*human render*), visualisasi ini dapat memperlambat proses pelatihan, sehingga bisa dipertimbangkan untuk menonaktifkan tampilan selama pelatihan dan hanya mengaktifkannya saat evaluasi. Variasi parameter (gamma, epsilon decay, learning rate, batch size) penting untuk memahami sensitivitas model dan meningkatkan performa tiap environment tugas.

6. Data dan Output Hasil Pengamatan

Pada praktikum ini setelah dilakukan pengujian terhadap *MountainCar-v0* dan keseimbangan tiang CartPole-v1 didapatkan hasil sebagai berikut:

No	Variable	CartPole-v1	MountainCar-v0
1.	Waktu Pelatihan	Pada CartPole-v1 relatif cepat mencapai performa yang lebih baik dalam beberapa ratus episode.	Pada MountainCar-v0 membutuhkan banyak episode agar lebih stabil.

2.	Jumlah Episode	Ditraining 1000 episode <pre>agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 32</pre>	Ditraining 1000 episode <pre>agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 32</pre>
3.	Skor per Episode (Testing)	Skor Rata rata meningkat signifikan (contoh: 8,9,10 hingga >200) <pre>Episode 4: Score = 9 Episode 5: Score = 8 Episode 6: Score = 10 Episode 7: Score = 8 Episode 8: Score = 10 Episode 9: Score = 10 Episode 10: Score = 10 Episode 11: Score = 10 Episode 12: Score = 10 Episode 13: Score = 9</pre>	Skor awal mencapai 199 karena maksimal langkah dalam satu episode 200 <pre>OUTPUT ... Filter Episode: 686/1000, Score: 199, Epsilon: 0.03 Episode: 687/1000, Score: 199, Epsilon: 0.03 Episode: 688/1000, Score: 199, Epsilon: 0.03 Episode: 689/1000, Score: 199, Epsilon: 0.03 Episode: 690/1000, Score: 199, Epsilon: 0.03 Episode: 691/1000, Score: 199, Epsilon: 0.03 Episode: 692/1000, Score: 199, Epsilon: 0.03 Episode: 693/1000, Score: 199, Epsilon: 0.03 Episode: 694/1000, Score: 199, Epsilon: 0.03 Episode: 695/1000, Score: 199, Epsilon: 0.03</pre>
4.	Kinerja Agen	Agen semakin stabil dengan skor yang makin tinggi.	Agen masih menghadapi tantangan untuk mencapai puncak bukit
5.	Kondisi Berhasil	Episode berakhir jika tiang jatuh atau batas langkah maksimum tercapai.	Episode berakhir jika mobil berhasil sampai puncak bukit (flag).
6.	Output		

7. Kesimpulan

Berdasarkan praktikum, analisis, dan hasil yang telah dilakukan, dapat disimpulkan bahwa:

- Praktikum Deep Q-Network (DQN) pada environment CartPole menunjukkan bahwa agen dapat belajar mengendalikan sistem secara efektif dengan memaksimalkan reward kumulatif.

- b. Parameter utama seperti γ , ϵ , dan learning rate sangat mempengaruhi kestabilan dan kecepatan pembelajaran.
- c. Pada CartPole-v1, agen relatif cepat mencapai kinerja optimal karena pola reward yang konsisten mampu mempermudah proses pembelajaran.
- d. Pada MountainCar-v0, agen membutuhkan jumlah episode yang lebih banyak serta strategi eksplorasi yang lebih baik untuk mencapai tujuan.
- e. Tantangan utama adalah menjaga keseimbangan eksplorasi dan eksploitasi serta stabilitas pelatihan jaringan saraf.

8. Saran

- 1. Menerapkan teknik target network untuk mengurangi ketidakstabilan pelatihan dan overestimation pada nilai Q. Hal ini dapat membantu model konvergen lebih stabil dan cepat.
- 2. Memperluas kajian dengan membandingkan algoritma RL alternatif, seperti Policy Gradient, Actor-Critic, maupun Proximal Policy Optimization (PPO). Perbandingan tersebut akan memberikan gambaran lebih jelas mengenai keunggulan dan keterbatasan relatif tiap algoritma, terutama pada lingkungan yang memiliki tingkat kompleksitas lebih tinggi.
- 3. Membuat waktu yang lebih optimal dengan pelatihan yang lebih baik.
- 4. Transfer learning dari model yang telah dilatih di lingkungan serupa dapat mempercepat proses konvergensi.

9. Daftar Pustaka

Ridho Amanda Putra. (t.t.). ANALISIS DEEP Q-NETWORK (DQN) UNTUK SIMULASI LAMPU LALU LINTAS ADAPTIF BERDASARKAN WAKTU TUNGGU KENDARAAN. 2024.

Sutton, R. S., & Barto, A. G. (t.t.). *Reinforcement Learning: An Introduction*.